2012 19th Asia-Pacific Software Engineering Conference

A CSP-theoretic framework of checking conformance of Business Processes

Suman Roy
Infosys Labs, Infosys Ltd.,
Bangalore 560 100, India
Email: suman_roy@infosys.com

Sidharth Bihary
Infosys Labs, Infosys Ltd.,
Bangalore 560 100, India
Email: sidharth_bihary@infosys.com

Jose Alfonso Corso Laos
Department of Industrial Engineering,
Pontificia Universidad Catolica De Chile,
340-Santiago-Chile
Email: pepecorso@gmail.com

***Abstract*—In this paper, we tackle the problem of conformance checking which verifies if the event logs (observed) match/fit the reference (arbitrary) process. We use concepts from Communicating Sequential Processes (CSP), which facilitates automated analysis using PAT toolkit. By this technique one can identify all the logs which cannot be properly replayed on the process. We illustrate our approach with an example. Finally, we introduce some metrics based on conformance checking. They are related to fitness, closeness, and appropriateness of the event logs vis-a-vis reference process models.**

***Keywords*-Formal methods, Software verification and validation, conformance checking, BPMN, Business Processes, CSP, PAT toolkit, Trace refinement.**

I. Introduction

Process-aware information systems comprising of work flow management etc are widely used in business these days as they provide precise descriptions of business processes. Business activities need to be monitored for auditing an organization (which is made mandatory by the new Sarbanes-Oxlay (SOX) Act [1]) in conjunction with business process modeling and simulation. Naturally one would like to ask this question how closely the monitored observations follow or fit the process. This is known as the problem of "*conformance*", *i.e.*, is there a proper fit between the real process (observed by the logs) and the original model?

Many information system, such as WFM (Work Force Management), ERP (Enterprise Resource Planning), and B2B (Business-to-business) systems maintain some kind of event logs (also called transaction logs, audit trails). A process log is represented by a set of traces/event sequences. Such logs, usually register the start and/or completion of activities. Moreover, each event refers to a case (process instance), an activity, and some additional data. On the other hand, processes are maintained in some form by all organizations. These process models are used for various purposes like, communication, ISO 9000 certification, system configuration analysis, simulation, requirements elicitation, test case generation etc and are referred to as reference processes. In this work, we adopt business processes models (compliant to BPMN) as reference processes which can be captured using an in-house Infosys tool, called InFlux. Furthermore, we have event logs (in MXML format) which have been recorded by deploying the processes in the field.

Under these assumptions we define a notion of conformance using traces. We say a process log conforms to a reference process model iff each trace or event sequence in the log can be replayed on the process, *i.e.*, the set of traces of the log is included in the set of traces for the process at hand. One way to check conformance of a process is to enumerate all finite traces (unraveling a loop only finite number of times if one such is present) in the process, and then carry out membership testing for each of the trace in the log. This checking takes quadratic time in terms of (maximum) length of the traces. However, the number of possible sequences generated by a process model may grow up exponentially, in particular for a model showing concurrent behavior. Hence, it might be quite expensive to list down all possible traces for the model. In this work, we tackle this conformance checking by choosing a proper formalism of Communicating Sequential Processes (CSP) [2] to represent these models for our purpose in order to avoid explicitly listing down all the traces. Communicating Sequential Processes (CSP) is a formal language for describing patterns of interaction in concurrent systems. The advantage gained by using these CSP models is that they can be readily fed into the PAT toolkit [3] for automated analysis like refinement checking. It can be shown that conformance checking problem is equivalent to trace refinement checking between the generated CSP processes.

We consider process logs and translate them to trace-equivalent CSP processes using prefix and external choice operators. We also propose an algorithm (an adaptation of an algorithm for generating CSP expressions for UML diagram originally proposed in [4]) for mapping a reference business process to an appropriate CSP process preserving traces. These CSP processes are fed into PAT model checker [3], [5] for conformance checking. We have implemented this technique on the top of PAT toolkit which we discuss through a running example. Moreover, using our technique it is possible to identify all the traces in the log which cannot be replayed on the process. An abridged version of this paper was published in [6] wherein we dealt with conformance

This work was done when Jose Corso was an intern with Infosys Labs, Bangalore during Dec'11-Feb'12.

1530-1362/12 $26.00 © 2012 IEEE
DOI 10.1109/APSEC.2012.110

checking of a particular class of business processes called structured processes (a structured process is a process where each XOR-split has a corresponding XOR-join and each AND-split has a corresponding AND-join) [7], whereas in this work we consider arbitrary/unstructured business processes.

The paper is organized as follows. We quickly dispense the preliminary concepts in Section II. A framework of process conformance is introduced in Section III. Next we discuss a mapping technique to translate a business process to a trace preserving CSP expression in Section IV followed by mapping of an event log to a CSP process in Section V. We describe conformance checking in detail in Section VI. Experimental results are discussed in Section VII. In section VIII we discuss metrics related to conformance checking. Finally we conclude in Section X which is preceded by a comparison of this work with other related work in Section IX.

II. Preliminaries

In this section we give brief overviews of some important concepts that we would be using later.

A. Process Logs

The log entries that we consider contain only event type, hence we drop event from the log entries in subsequent discussions (similar convention is followed in ProM-framework [8]). Let us assume T to be the set of log events (they are also called tasks/activities). We denote a trace as $\sigma \in T^*$, where $\sigma = t_0, t_1, \ldots t_{n-1}$, such that $t_i \in T, 0 \leq i \leq n-1$. We also denote the length of the trace $l = |\sigma| = l = n$ (that is, the number of tasks appearing in the trace). A process log, denoted as $W \in \mathcal{P}(T^*)$, is defined as a set of traces. The length $\zeta(W)$ of a process log W is the maximum of the length of all traces appearing in the log.

B. An overview of business processes

The Business Process Management Initiative (BPMI) has come out with a standard Business Process Modeling Notation (BPMN) for capturing pictorial representation of business processes. BPMN defines a Business Process Diagram (BPD) which is based on flowchart related ideas, and provides a graphical notation for business process modeling using objects like nodes, edges etc. We adopt a similar business process diagram based on a standard definition of business processes which consist of nodes like, events, activities, gateways and control flow relation linking two nodes. A *node* can be a task (also called an activity), an event, a fork (`AND-split`), a choice (`XOR-split`), a synchronizer (`AND-join`), a merge (`XOR-join`) gateway etc. In a process diagram, there are *start events* denoting the beginning of a process, and *end events* denoting the end of a process. A business process is *well-formed* if it has exactly one start node with no incoming edges and one outgoing edge from it, there is only one incoming edge to a task and exactly one outgoing edge from a task, each fork and choice has exactly one incoming edge and at least two outgoing edges, each synchronizer and merge has at least two incoming edges and exactly one outgoing edge, every node is on a path from the start node to some end node, and there exists at least one task in between two gateways (this is to avoid triviality). Unless otherwise mentioned, from now on, we shall consider only well-formed business processes.

The semantics of control elements of a business process are similar to that of work-flows discussed in [9]. If all the incoming branches of a join-choice are active at the same time, the semantics may cause correctness problems. There are two typical control flow related errors that can take place in processes: deadlocks and lack of synchronization. A *deadlock* implies that the process will never terminate, for instance when a choice is connected with a synchronizer. A *lack of synchronization* allows multiple instances of the same task to occur in a process, *i.e.* a fork is connected with a merge. Multiple instances can lead to some undesirable results, such as redundant activities, competition for resources, and dangling activities (e.g. one instance is synchronized with a task, and then the other instances are left dangling). A business process is *sound* if it does not produce deadlock and lack of synchronization. For obvious reasons, we shall consider only sound processes for conformance checking purposes. Moreover, there are standard techniques to check the soundness of processes, for example see [10].

Given a business process $\mathbf{P}$ we can define an *execution path* exp as a sequence of nodes, such that two consecutive nodes belong to the control flow relation, and exp begins and ends at the unique start and a final event respectively. However, we will be interested in execution paths restricted to tasks only. A *valid trace* τ of a business process $\mathbf{P}$ is a projection of an execution path on the set of activities/tasks, only keeping the start node at the beginning, and the end node at the end. The set of all valid traces of a process $\mathbf{P}$ is denoted as $\mathbb{T}_{\mathbf{P}}$.

C. An Introduction to CSP

In this section we present a brief introduction to *Communicating Sequential Process* (*CSP*) which is a process algebra for describing processes or programs. For further reading the reader is referred to [2], [11]. In CSP, a process depicts a kind of behavior, a behavior is made of events which are atomic and asynchronous between the environment (which can be another process) and the process. Processes communicate via events, they can be made compound by using the dot operator '$\cdot$'. We assume Σ to be a finite set of events.

The simplest process is $STOP$; this is a deadlocked

process that cannot communicate with its environment. The process $SKIP$ stands for successful termination followed by deadlock. The next one is a *prefix process* represented as $a \longrightarrow P$. The prefix process offers to engage in the event a and subsequently behaves like process P. The *generalized prefix process* is represented by $a : A \longrightarrow P(a)$ and it offers to engage in any event $a \in A$ and subsequently behave as process $P(a)$. The *external choice operator* written as, $P_1 \ \Box \ P_2$, offers to behave either as P_1 or as P_2 at the choice of the environment. On the other hand, the *internal choice operator* represented as $P_1 \sqcap P_2$ may behave either as P_1 or as P_2 in a non-deterministic fashion. The latter choice between P_1 and P_2 is resolved independently of the environment. The *alphabetized parallel operator*, written as $P_1 \underset{B}{\parallel} P_2$ is a parallel composition of P_1 and P_2 where P_1 and P_2 have to synchronize on all events in B and behave independently with respect to all other events. There is an *interleaving operator* in CSP which says that the processes which are interleaved may behave in an independent manner with respect to all the events except successful termination. Interleaving is represented as $P_1 \ ||| \ P_2$. Conditional choices can also be presented in CSP, *e.g.*, if b then P_1 else P_2, where b is a Boolean condition. It is abbreviated as $P_1 \not< b \not> P_2$. The process $P \setminus A$, (the symbol '\' is referred to as the *hiding operator*) behaves like the process P except that in the behavior of P, events in A are hidden.

A *trace* of a CSP process is a sequence of visible events that the process can perform. The traces model is a non-empty and prefix closed set of such traces. Let the set of traces of process P be called $traces(P)$. In the traces model, the processes $P_1 \Box \ P_2$ and $P_1 \sqcap P_2$ are considered to be equivalent, that is, $traces(P_1 \Box \ P_2) = traces(P_1 \sqcap P_2)$. An empty trace is denoted as $\langle \{ \} \rangle$.

Refinement Relations: The main idea of refinement is that an implementation behaves correctly when its behavior meets the behavior of the specification (its denotational semantics defines both what a process can do and what a process must do). When the implementation is indeed correct we can say that the implementation refines the specification. We only mention the definition for trace refinement relation for traces models. Given an implementation (CSP) process $IMPL$ and a specification (CSP) process $SPEC$ we say "implementation trace refines specification" if and only if

$$SPEC \sqsubseteq_{\mathcal{T}} IMPL \Leftrightarrow traces(IMPL) \subseteq traces(SPEC)$$

PAT tool: Process Analysis Toolkit (PAT) is a self-contained framework [3] which provides the functionalities for composing, simulating and reasoning of concurrent, real-time systems and other possible domains. It comes with user friendly interfaces, featured model editor and animated simulator. Most importantly, PAT implements various model checking techniques catering for different properties such as deadlock-freedom, divergence-freedom, reachability, LTL properties with fairness assumptions, refinement checking and probabilistic model checking. There is also an advantage of using PAT in an industrial setting as it can be used on windows environment.

No of instances	Log Traces
16	ABCEGDFHIJKON
29	ACGFBEDHIJKOM
50	ACBGFDEHIJKON
68	ABCFGDEHIJKOM
79	ACBFGDEHIJKON
4	ABECDFGHIJKON

(a) Event Log 1

No of instances	Log Traces
40	ACBEGFDHIJKOM
26	ACBGFEDHIJKOLJKOM
94	ABCDEGFHIJKOLJKOM
75	ABECFGHDIJKON
89	ACBDEGFHIJKOLJKON
49	ABCEGDFHIJKOLJKOLJKON

(b) Event Log 2

No of instances	Log Traces
92	ACBEGFDHIJKOL
34	ACBGFEDHIJKOLJKOM
81	ABCDEGFHIJLKOJKOM
9	ABECHFGDIJKON
80	ACBDEGFHIJKOLJKON
2	ABCEGDFHIJKOLJKOLJKON

(c) Event Log3

Table I
EVENT LOGS FOR THE RUNNING EXAMPLE

D. Running example

We shall use an example of a process of opening an account in bank (*cf.* Figure 1). In this process the bank receives request (A) for opening an account from a client. The bank then obtains client information (B) and selects service (C). Getting client information reduces to getting driver's license (D) and analyzing customer relationship (E) whereas select service forks into record information (F) and submit deposit (G). After analysing customer relation and recording information, document is prepared (H) followed by preparing for account opening (I). For final phase of account opening, a review process is scheduled (J) which follows account status review (K) and ID verification (O). Finally there is a decision where either a request is rejected (M), or an account is opened successfully (N), or missing data is sought (L) for. Note that the activities are marked using capital letters in parentheses corresponding to the activities in the diagram.

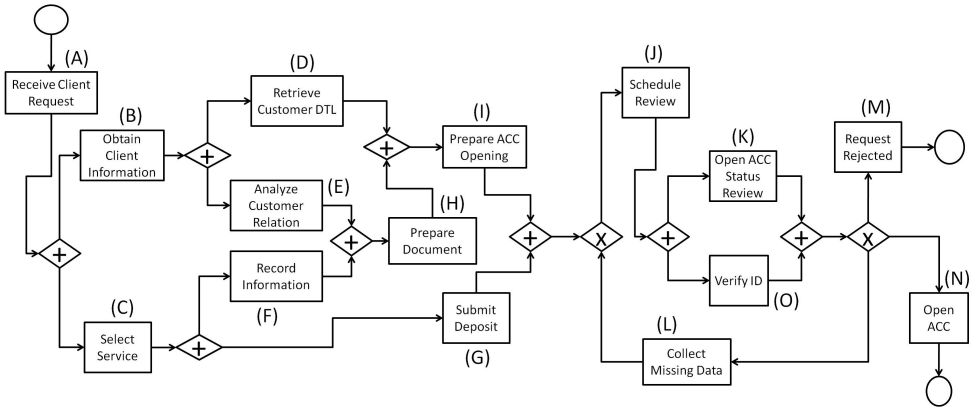


Figure 1. An example of a bank account opening process

III. A framework of Process conformance

Given a process log W over a set T of log events, and a reference model $\mathbf{P}$, we say that the process log W conforms to process model $\mathbf{P}$ (written as $W \sqsubseteq_C \mathbf{P}$) if all the activities in W occur in $\mathbf{P}$ also, and τ is a trace in W implies τ is a trace in $\mathbf{P}$, *i.e.*, $W \subseteq \mathbb{T}_{\mathbf{P}}$. Similarly, one can define whether a process model P conforms to a process log W (written as $\mathbf{P} \sqsubseteq_C W$). These problems are known as conformance checking (ConCheck) problem. To decide the ConCheck problem we need to perform membership testing problem. That is, given an arbitrary process log W and a fixed (reference) BPM model $\mathbf{P}$ we would like to check whether any trace in W is also a trace in $\mathbf{P}$. For a fixed business process it is possible to enumerate all possible traces up to a fixed length. Take this length to be the maximum length of traces in the event log. As this is a one-time construction and our input is an arbitrary trace and a fixed process, the inclusion problem can be decided in time quadratic in the maximum length of traces. However, the number of traces created by a process model may grow exponentially, specially when a process model has AND-gateways and shows concurrent behavior. For example, there are $5! = 120$ possible ways of executing 5 parallel tasks and $8! = 40320$ possible combinations for 8 parallel tasks. Thus it may be quite expensive to enumerate a large number of traces of the model for checking the membership. Further, in an industrial scenario, we needed the help of an off-the-shelf tool (model checker) that could settle the conformance checking problem quite easily and efficiently so that the end user does not need to know much about the technique. This is the reason we take an intermediate step of converting these models to trace equivalent CSP descriptions such that the conformance checking problem can be reduced to trace refinement problem, and can be solved using PAT model checker.

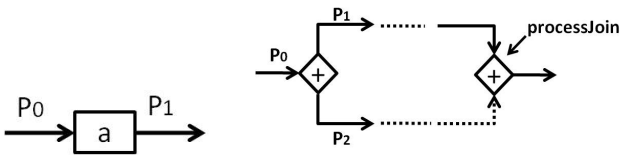
IV. Mapping business process to CSP Processes

In [12] Van der Aalst *et.al* introduced few workflow patterns, known as the 'gold standard' benchmark of workflow language. They vary from very simple patterns (*e.g.,* sequential routing) to complex patterns (*e.g.*, synchronization merge). Wong *et al.* proposed a translation of these patterns to CSP descriptions [13] in an attempt to specify and refine workflow processes. Using the behavioral model of CSP they were able to develop workflow in a compositional manner. However, this technique works for only structured process models as documented in [6]. For arbitrary processes, we adopt a different strategy which is a modification of the algorithm used for converting UML diagrams to CSP descriptions [4]. The basic methodology of the mapping is to decompose the process model into atomic patterns and generate independent CSP description for each of the patterns. Then using the prefix process the CSPs are merged to get the complete CSP description of the whole process. Activity and event nodes correspond to simple prefix processes. For gateway nodes it uses different operators depending upon the type of the gateway. The nodes of the process model are considered as events in the CSP description and the edges of the model are treated as processes. We use the event $event.a$ to represent the completion of activity *'a'* associated with the CSP. The CSP is generated in a top-down fashion from the model starting from the start event of the process to an end event.

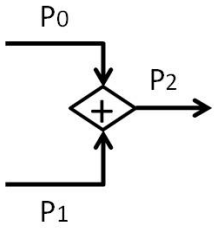
A. Basic Control Flow Patterns

We consider basic control flow patterns [12] of business processes and translate them to corresponding trace-preserving CSP processes.

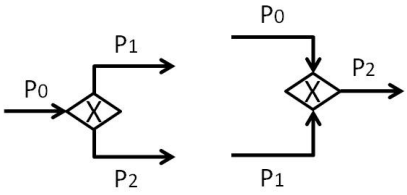
Start event : It denotes the beginning of a process. A CSP event *start* is defined for the corresponding start event in the reference process. The process P_0 denotes the primary CSP for the whole reference process. The pattern is modeled in CSP as



(a) A single activity (b) An AND-split/fork



(c) An AND-join/synchronizer



(d) An XOR-split/choice (e) An XOR-merge

Figure 2. Different kinds of patterns in business processes

$$P_0 = start \longrightarrow P_1$$

End event : It denotes the successful termination of the process model. It is captured by ***SKIP*** in CSP.

$$P = SKIP$$

Activity: The execution of an activity a is denoted by $event.a$. In Figure 2(a) let P_0 denote the behavior of the prefix process associated with the edge into activity a. The mapping assumes that P_0 executes $event.a$ and then behaves like the process P_1. P_1 again corresponds to the process on the outgoing from edge from a. This convention will be followed throughout for generating CSP expressions for other patterns of a business process.

$$P_0 = event.a \longrightarrow P_1$$

Parallel split(AND-split): To ensure that the parallel processes originating at AND-splits are synchronized at appropriate AND-joins, we use alphabetized parallel operator. At each parallel split, all the AND-joins are identified where these parallel processes merge. These AND-joins are specified as alphabets associated with the parallel operator so that the concurrent processes synchronize on them.

$$P_0 = P_1 \ |[processJoin]| \ P_2$$

Synchronizer (AND-join): The behavioral property of the synchronizer states that the process can proceed only after the end of the execution of all the input activities. For example, in Figure 2(c) P_2 can be executed only after both P_0 and P_1 are executed. To capture this, all the input processes of the AND-join are synchronized on an event, let us call it *processJoin* (associated with the AND-join). As this event synchronizes all the concurrent processes, it will block until all the input processes participate in the execution. After all the input processes complete their execution, one of the input processes will proceed further representing the output process and all other are used for synchronization, and will eventually lead to successful termination.

$$P_0 = processJoin \longrightarrow P_2,$$
$$P_1 = processJoin \longrightarrow P_1$$

Exclusive choice (XOR-split) : This operator becomes enabled when the input activity is executed and subsequently, one of the output activities is triggered (see Figure 2(d)). The choice is guarded by some conditions which can be evaluated by an external environment or by an internal system depending upon the situation. The conditions associated with the choice operator should be mutually exclusive or else it will give rise to a race condition among the processes. We model this case by the following CSP expression.

$$P_0 = P_1 \not< cond \not> P_2$$

Exclusive merge (XOR-join) : If one of the incoming edges is executed, an XOR-merge fires leading to the execution of an appropriate activity on its outgoing edge, see Figure 2(e).

$$P_0 = P_2, \ P_1 = P_2$$

Some other control flow patterns can be defined likewise. For example, an inclusive-OR split of activity a into two activities a_1 and a_2 can be modeled with an XOR-split with three outgoing flows containing a_1, a_2 and another flow with AND-join of a_1 and a_2 etc. Using the laws for traces [11] like $traces(STOP) = \{\langle\rangle\}, traces(a \longrightarrow P) = \{\langle\rangle\} \cup \{\langle a\rangle^\frown s \| \ s \in traces(P)\}$ followed by pushing the hiding operator inside CSP expressions [11] one can prove the following:

Proposition 1: Let Q be the CSP process generated from an arbitrary process $\mathbf{P}$ following the algorithm above. Let $\mathcal{J}$ denote the set of additional events introduced corresponding to synchronizers (AND-joins) in the process. Then $traces(Q \setminus \mathcal{J}) - \langle\{\ \}\rangle = \mathbb{T}_{\mathbf{P}}$.

Now let us consider the process for the running example in Figure 1. Let the CSP process P_0 describe the behavior of the whole process and $P_1, P_2, \ldots$ denote the intermediate CSP processes. They are further described in Listing 1.

V. A CSP description of Event logs

The CSP description for an event log is generated by constructing a single CSP process for each trace in the log set and then aggregating all of them using external choice operator. The CSP for each trace is a simple prefix process as the traces contain only the sequence of activities. Consider an example of an arbitrary process log $W = \{ABCD, ACBD, ABD\}$. A corresponding CSP

Listing 1 A CSP description for the process in Figure 1

$$
\begin{aligned}
P_0 &= start \longrightarrow P_1 \\
P_1 &= event.RecieveClientRequest \longrightarrow P_2 \\
P_2 &= P_3 \ |[processJoin.J1, processJoin.J2, processJoin.J3]| \ P_4 \\
P_3 &= event.ObtainClientInformation \longrightarrow P_8 \\
P_4 &= event.SelectService \longrightarrow P_5 \\
P_8 &= P_9 \ |[processJoin.J2]| \ P_{10} \\
P_5 &= P_6 \ |[processJoin.J3]| \ P_7 \\
P_9 &= event.RetriveCustomerDTL \longrightarrow P_{11} \\
P_{10} &= event.AnalyseCustomerRelation \longrightarrow P_{12} \\
P_6 &= event.RecordInformation \longrightarrow P_{13} \\
P_7 &= event.SubmitDeposit \longrightarrow P_{14} \\
P_{12} &= processJoin.J1 \longrightarrow P_{15} \\
P_{13} &= processJoin.J1 \longrightarrow P_{13} \\
P_{15} &= event.PrepareDocument \longrightarrow P_{16} \\
P_{11} &= processJoin.J2 \longrightarrow P_{17} \\
P_{16} &= processJoin.J2 \longrightarrow P_{16} \\
P_{17} &= event.PrepareAccOpening \longrightarrow P_{18} \\
P_{18} &= processJoin.J3 \longrightarrow P_{19} \\
P_{14} &= processJoin.J3 \longrightarrow P_{14} \\
P_{19} &= P_{20} \\
P_{30} &= P_{20} \\
P_{20} &= event.ScheduleReview \longrightarrow P_{21} \\
P_{21} &= P_{22} \ |[processJoin.J4]| \ P_{23} \\
P_{22} &= event.OpenACCStatusReview \longrightarrow P_{24} \\
P_{23} &= event.VerifyID \longrightarrow P_{25} \\
P_{24} &= processJoin.J4 \longrightarrow P_{26} \\
P_{25} &= processJoin.J4 \longrightarrow P_{25} \ ((processJoin.J4)^{\zeta(W)} \longrightarrow SKIP) \\
P_{26} &= P_{28} \not< passes \not> P_{29} \not< needFurtherInfo \not> P_{27} \\
P_{28} &= event.OpenAccount \longrightarrow P_{32} \\
P_{29} &= event.CollectMissingData \longrightarrow P_{30} \\
P_{27} &= event.RequestRejected \longrightarrow P_{31} \\
P_{31} &= SKIP \\
P_{32} &= SKIP
\end{aligned}
$$

description of the log is as follows.

$$
\begin{aligned}
Q_W = {} & start \longrightarrow event.a \longrightarrow event.b \longrightarrow event.c \\
& \longrightarrow event.d \longrightarrow SKIP \\
& \Box \ start \longrightarrow event.a \longrightarrow event.c \longrightarrow event.b \\
& \longrightarrow event.d \longrightarrow SKIP \\
& \Box \ start \longrightarrow event.a \longrightarrow event.b \longrightarrow event.d \\
& \longrightarrow SKIP
\end{aligned}
$$

We assume each of the traces will begin with the *start* event, although it cannot be observed (which denotes the initialization of the process instance corresponding to the trace). It is easy to see that traces of Q_W (modulo the empty trace) coincide with the original traces in W. This along with Proposition 1 shows that the conformance checking for processes reduces to trace refinement checking of generated CSP processes.

VI. Conformance checking and error logs

We have implemented our approach in two steps. In one step we take event logs as input and generate a simple CSP process Q_W as discussed in Section V, which is called implementation $Impl$. In the next step, we consider the reference process $\mathbf{P}$ and translate it into an appropriate CSP description $Q_{\mathbf{P}}$, called specification $Spec$, using our technique described in Section IV. Both these CSP models are fed into PAT tool. We check whether $Impl$ trace refines $Spec$, which decides if the process log conforms to the reference process. If it does not, the tool reports the shortest counterexample. When we perform the conformance of process logs in Table I(c) against the reference process in Figure 1. PAT tool generates the shortest counterexample: "$start \longrightarrow event.a \longrightarrow event.b \longrightarrow event.e \longrightarrow event.c \longrightarrow event.h$", that is the sequence of activities A, B, E, C, H which is not a prefix of any trace in the reference process. Because, necessary documents for account opening cannot prepared immediately after a service is selected, see Figure 1. We define the *error trace*[1] to be an element in an event log which has a prefix as a counterexample trace. One can thus find all the error traces that have the above counterexample trace as a prefix. Delete this set of error traces from the log and perform one more round of conformance checking. If another counterexample trace is detected then find all the error traces having the former as a prefix and continue. Thus all the error traces in W can be found constructively. Check that one can obtain the list of error traces as $\mathcal{E}_W = \{ACBEGFDHIJKOL, ABCDEGFHIJLKOJKOM, ABECHFGDIJKON\}$ in this case. In a certain situation a prefix of the shortest counterexample trace can be replayed on the reference process model up to the node preceding an end node. In that the error trace corresponding to this counterexample trace will be its prefix that ends at a node preceding an end node of the reference process.

We remark that we could find an interesting limitation of PAT/FDR toolkit during this exercise. These tools cannot handle parallel composition within a recursive process as found in the example of Figure 1. That is the reason we change the CSP process description for a synchronizer lying inside a loop. Assume $\zeta(W)$ to be the length of a process log W (see Section II-A). If the join in Figure 2(b) appears within a loop then the CSP processes would be changed to, $P_0 = processJoin \longrightarrow P_2, P_1 = processJoin^{\zeta(W)} \longrightarrow SKIP$, where $P^2 = P \longrightarrow P$ etc. Correspondingly, the CSP description for the process in Figure 1 is changed in Listing 1. Another point to note is that when we want to

[1]we do not handle exceptions in this work

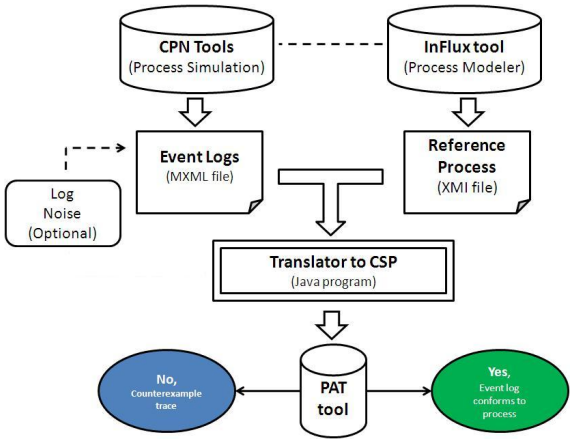


Figure 3. An experimental framework for checking conformance

check the other side of the trace refinement, *i.e.,* if the process model conforms to the log, the tool gets stuck as it cannot handle infinite traces for refinement checking.

VII. Experimental Results

We have a built a tool chain for conformance checking around PAT tool. We use an in-house process modeler (InFlux tool) to model the process which generates an XMI file. The event logs are assumed to be in MXML format. We have written Java code to convert XMI representation of processes to CSP descriptions in machine readable format of PAT toolkit. Similarly, another Java code is used to translate event logs to machine readable CSP description. Both these CSP expressions are fed into PAT model checker to check for refinement. The tool answers in the affirmative if conformance check succeeds. Otherwise, it reports the shortest counterexample trace. A schematic diagram of this setup is shown in Figure 3. We carry out some experiment related to conformance checking on industrial process models. We use a standard algorithm [14] to convert a process to a trace equivalent Petri net and use CPN tool [15] to generate traces, which is later projected into the activities of the original process to obtain the set of final traces. The tool allows one to introduce spurious/additional logs (noise) in the generated log files. For experimentation purposes, we use Intel Pentium 4 CPU, 2.8 GHz, a physical memory of 2 GB RAM on Microsoft windows XP Service Pack 3.

The business processes which are modeled using InFlux tool arise in different business domains, they are segregated into three libraries A, B, and C depending on the domain. Lib A, B and C contain (13+7), 12, 18 models thus aggregating 50 models. We furnish details of static data of our process libraries in Table II. We capture the number of nodes (include activities, gateways, start and end events), the number of edges between these nodes, number of AND and XOR gateways etc. Next we generate sets of traces for each of the processes using technique discussed above. Now

Libraries	A	B	C
No of processes	20	12	18
Avg/max no of nodes	15.6/20	27.21/75	24.14/34
Avg/max no of edges	11.6/16	22.92/72	20.42/30
Avg/max no of AND gates	0/0	0.285/2	0.714/2
Avg/max no of XOR gates	1/1	23.57/65	31.85/54
Avg/max no. of traces in the log set	23.6/36	23.57/65	31.85/54
Length of the longest trace	11	34	11

Table II
STATIC DATA FOR PROCESS LIBRARIES

we generate CSP descriptions for them to run PAT tool for conformance checking. The experimental results are given in Table III. The time taken for conformance checking is negligible, for most of the real life processes conformance checking can be performed quickly.

Libraries	A	B	C
Avg no of explored states	38.6	56.78	72
Max no of explored states	84	136	110
Avg length of error trace	1.4	1.28	0.85
Avg analysis time (in seconds)	0.040	0.034	0.044
Total analysis time (in seconds)	0.807	0.414	0.796

Table III
CONFORMANCE CHECKING OF INFLUX PROCESSES USING PAT

VIII. Metrics for conformance checking

Conformance checking can be measured in terms of metrics which portray how closely the observed sequences follow the business processes [8], [16]. However, we use a different framework to formulating such metrics which we describe in some detail below.

Fitness metric

There can be several ways to measure the fitness between event logs and process models. One possibility is to replay an error log in the model and somehow reconstruct a valid trace of the process and see how far it deviate from the former. The aim is to find out how well the deployed process match the reference process. Let us consider a counterexample trace τ corresponding to a process log (as constructed in Section VI). Let $\mathcal{E}_W^\tau \subseteq \mathcal{E}_W$ be the set of error traces in log W which have a prefix trace as τ. The trace segment τ is replayed in the CSP for the reference model using ProBE tool. As ProBE is an interactive simulation tool, the counter example can be replayed in the reference model just before the error element and in order to find the actual trace corresponding to the counter example, the replay can be continued along the possible paths shown in the tool. Thus a valid trace τ' can be constructed from it. This valid trace may not be unique. However, we shall consider one which is the shortest among them. Let n be the aggregated number of traces in the log file, l_τ^c be the length of the counterexample trace τ, and l_τ^s be the length of the shortest valid trace reconstructed from τ described above. Further,

let f_i^τ be the frequency of the ith error trace in $\mathcal{E}_W^\tau$, and $\mathcal{T}$ be the set of all counterexample traces for the concerned log file. Then the fitness metric is given by,

$$\begin{aligned}\alpha &= 1 \quad \text{if there is no counterexample trace}\\ &= 0 \quad \text{if a prefix of a counterexample trace is also a valid trace in the process excld the relevant end node}\\ &= \frac{\sum_{\tau\in\mathcal{T}}\sum_{i\in\mathcal{E}_W^\tau} f_i^\tau(|l_\tau^s - l_\tau^c|/l_\tau^s)}{n} \quad \text{otherwise}\end{aligned}$$

Observe that we force some boundary conditions on the metric and also ensure that the metric function is monotonically decreasing with respect to $\frac{l_\tau^c}{l_\tau^s}$. Specifically we make α to be 0 when a prefix of a counterexample trace can be replayed on the reference process from the start node to a node just preceding an end node.

Closeness metric

Another interesting metric could be related to the amount of closeness of the event log W with the process model. It should measure the fraction of traces in the log that can be replayed on the process model. This can be captured by a simple metric as follows, as it is possible to generate the list of error traces in W. This closeness metric β can range from 0 to 1. Notably, it is 1 when there is no counterexample trace. Using the notations used for the earlier metric,

$$\beta = \frac{n - \sum_{\tau\in\mathcal{T}}\sum_{i\in\mathcal{E}_W^\tau} f_i^\tau}{n}$$

Appropriateness metric

The metric related to appropriateness of the process log can be subjective, and is defined to suit the purpose. Our definition of appropriate metric is centered around process flow related ideas [8], [16]. The aim of formulating such a metric is to see how clearly the model reflects the behavior observed in the log. The degree of appropriateness will depend on both the structural and behavioral aspects of it.

Structural appropriateness It is possible to associate a metric with respect to the size of a business process. It might indicate how compactly and meaningfully the process has been modeled. It is desirable for the metric to have the following property, the higher the value the better the quality of the model is. Also, the metric should not exceed the value of 1. A business process has two dedicated nodes, designated as start and end nodes, while each gateway and activity correspond to a node separately. Let $\mathcal{A}$ be the set of activities/tasks in the process and N be the set of nodes including (start and end) events, activities (excluding dummy activities) and gateways. The structural appropriate metric can be defined as

$$\zeta = \frac{|\mathcal{A}| + 2}{|N|}$$

Behavioral appropriateness It is possible to measure appropriateness with respect to the behavior observed in the log. Even if the log fits the model, there might be some behavior present in the model, but has not been observed. In fact, when the model shows too many behavior it becomes less informative in describing the process. Following similar ideas from [8], [16], we proceed to quantitatively measure the possible behavior reflected in the log by determining the mean number tasks enabled during the log replay using ProBE tool, with the hope that an increase of potential behavior will result in a higher number of transitions being enabled during log replay.

Let us formulate the metric now. Let n denote the aggregated number of traces, m denote the number of instances in the log, $|\mathcal{A}|$ the total number of activities in the process model, f_i the frequency of the ith trace in the log (note $n = \sum_i^m f_i$), and μ_i the mean number of tasks enabled during the replay of the ith trace by ProBE (given as the ratio of the number of tasks enabled on ProBE during the simulation of the trace and the length of the trace). The behavioral appropriateness metric is given by

$$\eta = 1 - \frac{\sum_{i=1}^m f_i(|\mu_i - 1|)}{(|\mathcal{A}| - 1)n}$$

For illustrating the usefulness of metrics, we again use the reference process model in Figure 1 and event logs in Tables I(a),I(b) and I(c). Table IV shows different values for the metrics for event logs 1, 2 and 3 *wrt* the reference process. From the values of fitness metric, event log 1 and event log 2 fit perfectly with the reference model as there is no error trace present in them. Though most of the traces from event log 3 do not conform to the reference model, the fitness value still comes out to be 0.886, because in case of a counter example, it measures how close the counter example is with the correct behavior of the reference model. The counter examples for event log 3 are small compared to their corresponding (reconstructed) shortest valid traces, so we get fitness metric close to 1. Closeness metric values shows the percentage of the instances in the event log, that actually conform to the process - in this case only 38.9% of the instances in event log 3 conform to the reference model. As we deal with only one reference model, the structual appropriateness metric is same for all the event logs. Note this metric is independent of the event logs and only measures the appropriateness of the model. Behavioral metric measures the degree of parallelism and alternative flows in the reference model *wrt* the event logs. Though both event log 1 and event log 2 fully conforms to the reference model, they have different behavioral appropriateness values, which denote that the traces in event log 1 are behaviorally closer to the model compared to the traces of event log 2. Besides a more interesting observation is the appropriateness metric value for event log 3 which is the largest among all the values. This is because, even though event log 3 has a couple of error traces, the remaining correct traces are behaviorally closer to the reference model.

Metrics	Event log 1	Event log 2	Event log 3
α (fitness)	1	1	0.886
β (closeness)	1	1	0.389
ζ (struct. appropriateness)	0.607	0.607	0.607
η (behav. appropriateness)	0.988	0.986	0.993

Table IV
DIFFERENT METRIC VALUES

IX. A Comparison with related work

The process conformance checking problem that we deal with is very similar to the work of Cook *et al.* in [17], [18]. A software validation method is introduced in [17], in which event streams coming from the model and real-life observations are compared based on two different string distance metrics. Cook *et al.* developed techniques for process validation that could uncover and measure the discrepancies between models and executions [19]. They considered a process execution and a process model, and measured the level of correspondence between the two using string distance metrics.

As conformance checking aims at detecting inconsistencies between a process model and event log, validation can be quantified by formulating and measuring suitable metrics. In [8], the authors have proposed metrics for similar conformance checking problems. They introduce a criterion called, fitness, by which they measure how many traces in the log events can be played on the reference model. They have proposed another metric, called appropriateness which captures the structural and behavioral similarities between the event logs and process model. In [16], the authors have extended this work, dealt elaborately on the formulation of metrics, and finally evaluated them on artificial and real-life event logs. In another article [20], van der Aalst *et al.* address the problem of conformance checking on service behaviors and estimate how much the actual behavior a service meets the expected behavior specified in the model using metrics of fitness and appropriateness. While our work assumes the existence of a business process as a reference process, these methods consider a Petri net as the underlying formalism for a process. Moreover, our work can answer the question whether process logs fit the reference model exactly. In fact, one can define similar metrics based on simulation of CSP models to address similar issues like fitness, appropriateness etc. Another advantage of our work is that we are able to detect all the traces which are not aligned with the reference process. A related conformance checking problem is addressed in [21] where a temporal (LTL) formula is model checked against a log using LTL checker available in ProM. The authors have reported a language to formulate properties in the context of event log that can store information about activities, cases, timestamps, originators, and related data. However, their work requires some human intervention in the sense the properties have be to encoded in LTL before they are fed to the property checkers, whereas in our approach the end user does not need to know the intricate details of the checking procedure. One interesting extension of our work could be to use the ideas related to incorporating more features of event logs, like timestamp for conformance checking.

Additionally we remark on a comparison of our tool chain with ProM-framework [8]. Among various process mining tools available, ProM is one of the prominent one which comes with a 'Conformance Checker' plug-in. The methodology adapted by the tool is to replay the event logs in the reference process keeping track of the points where the trace violates the behavior of the reference process, simultaneously calculating the metrics discussed earlier. To denote the completion of the replay of every log, the tool supplies virtual tokens at each error point. Thus if the value of fitness metric is 1, then the log conforms to the process, otherwise not. But if the value of fitness metric turns out to be less than 1 and the process contains invisible/duplicate activities, then it is not always the case that the concerned trace does not fit the model, for ProM always chooses the Minimum Description Length (MDL) to enable the currently replayed task. There may be some other way (longer one) of replaying it without error, but ProM did not choose it. However, our approach does not need this heuristic and correctly finds the error traces for such cases.

Bergenthum *et al.* presented a polynomial algorithm in [22] to verify if a given scenario would be an execution of a Petri net. Normally, a scenario is given as a labeled partial order (LPO) over the set of possible actions (events) and abstracting from the conditions in a process gives an LPO, also called a run (a casual ordering of events). This approach might help us to find an efficient algorithm to directly check for the conformance of logs *w.r.t* the process. In the work [23] Ramos *et al.* proposed an approach for composition of frameworks thereby developing a reasoning strategy for behavioral conformance of the composed framework. Their definitions of conformance were based on the failure models of CSP and process refinement notions which is in contrast to an easier model of trace refinement that we employ for conformance checking.

X. Conclusion

We have proposed a novel method for checking conformance of business processes against event logs. The merit of our method is that one can directly check for process conformance and find out the error traces appearing in the event log. The advantage of the using process algebraic languages like CSP is that they offer well defined semantics, modularity, and extensibility. The lack of formal semantics associated with the gateway constructs of process models

has resulted in different interpretations by different vendors, thus significantly reducing the inter-operability of those models. A CSP description of those models might alleviate such problem. It could be interesting to use log data on a finer level of granularity (*e.g.*, including events like 'start', 'execute', 'completion' of activities) and to capture other perspectives such as data, time-stamp, and then find CSP descriptions for them for conformance checking. Although CSP description may lead to complex representations of business processes the end user does not have to see CSP, which can facilitate automatic analysis of business processes. This technique also helps identify all the error traces. Further, there is a lot of scope for developing new techniques for measuring and visualizing non-conformance.

References

[1] P. Sarbanes, G. Oxley, and et. al, "Sarbanes-oxley act of 2002," 2002.

[2] C. A. R. Hoare, *Communicating Sequential Processes*. Prentice Hall, 1985.

[3] J. Sun, Y. Liu, A. Roychoudhury, S. Liu, and J. S. Dong, "Fair model checking with process counter abstraction," in *Proceedings of the 2nd World Congress on Formal Methods*, ser. FM '09. Springer-Verlag, 2009, pp. 123–139.

[4] D. Bisztray and R. Heckel, "Rule-level verification of business process transformations using CSP," *ECEASST*, vol. 6, 2007.

[5] Y. Liu, J. Sun, and J. S. Dong, "PAT 3: An extensible architecture for building multi-domain model checkers," in *IEEE 22nd International Symposium on Software Reliability Engineering, (ISSRE'11)*, 2011, pp. 190–199.

[6] S. Bihary, J. Koneti, and S. Roy, "Process conformance using CSP," in *Proceeding of the 5th ISEC'12*. ACM, 2012, pp. 139–142.

[7] B. Kiepuszewski, A. H. M. t. Hofstede, and C. Bussler, "On structured workflow modelling," in *Proceedings of the 12th International Conference on Advanced Information Systems Engineering*, ser. CAiSE '00. London, UK: Springer-Verlag, 2000, pp. 431–445.

[8] A. Rozinat and W. M. P. van der Aalst, "Conformance testing: Measuring the fit and appropriateness of event logs and process models," in *Business Process Management Workshops*, vol. 3812, 2005, pp. 163–176.

[9] R. Liu and A. Kumar, "An analysis and taxonomy of unstructured workflows," in *Business Process Management*, ser. LNCS, W. M. P. van der Aalst, B. Benatallah, F. Casati, and F. Curbera, Eds., vol. 3649, 2005, pp. 268–284.

[10] D. Fahland, C. Favre, B. Jobstmann, J. Koehler, N. Lohmann, H. Völzer, and K. Wolf, "Analysis on demand: Instantaneous soundness checking of industrial business process models," *Data Knowl. Eng.*, vol. 70, no. 5, pp. 448–466, 2011.

[11] A. W. Roscoe, *The theory and practice of concurrency*. Prentice Hall, 1997.

[12] W. M. P. Van Der Aalst, A. H. M. Ter Hofstede, B. Kiepuszewski, and A. P. Barros, "Workflow patterns," *Distrib. Parallel Databases*, vol. 14, no. 1, pp. 5–51, 2003.

[13] P. Wong and J. Gibbons, "A process-algebric approach to workflow specification and refinement," in *Software Composition*, ser. Lecture Notes in Computer Science, vol. 4829, 2007, pp. 51–65.

[14] R. M. Dijkman, M. Dumas, and C. Ouyang, "Semantics and analysis of business process models in BPMN," *Inf. Softw. Technol.*, vol. 50, no. 12, pp. 1281–1294, 2008.

[15] A. Karla, A. De Medeiros, and C. W. Günther, "Process mining : Using CPN tools to create test logs for mining algorithms," in *Proceedings of the Sixth Workshop and Tutorial on Practical Use of Colored Petri Nets and the CPN Tools*, 2005, p. 177Ű190.

[16] A. Rozinat and W. M. P. van der Aalst, "Conformance checking of processes based on monitoring real behavior," *Information Systems*, vol. 33, pp. 64–95, 2008.

[17] J. E. Cook and A. L. Wolf, "Software process validation: Quantitatively measuring the correspondence of a process to a model," *ACM Transactions on Software Engineering and Methodology*, vol. 8, no. 2, pp. 147–176, 1999.

[18] J. E. Cook, , C. He, and C. Ma, "Measuring behavioral correspondence to a timed concurrent model," in *In Proceedings of the 2001 International Conference on Software Mainenance (ICSM)*, 2000, pp. 332–341.

[19] J. E. Cook and A. L. Wolf, "Discovering models of software processes from event-based data," *ACM Trans. Softw. Eng. Methodol.*, vol. 7, no. 3, pp. 215–249, 1998.

[20] M. van der Aalst, Wil M. P.and Dumas, C. Ouyang, A. Rozinat, and E. Verbeek, "Conformance checking of service behavior," *ACM Trans. Internet Technol.*, vol. 8, pp. 13:1–13:30, 2008.

[21] W. M. P. van der Aalst, H. T. de Beer, and B. F. van Dongen, "Process mining and verification of properties: An approach based on temporal logic," in *OTM Conferences (1)*, 2005, pp. 130–147.

[22] G. Juhás, R. Lorenz, and J. Desel, "Can I Execute My Scenario in Your Net?" in *ICATPN*, 2005, pp. 289–308.

[23] R. Ramos, A. Sampaio, and A. Mota, "Framework composition conformance via refinement checking," in *Proceedings of the 2008 ACM symposium on Applied computing*, ser. SAC '08. ACM, 2008, pp. 119–125.